

Unit 2.0 Function, Class & Object

2.1 Function: Call by value, Call by reference, Default arguments

2.2 Inline function

2.3 Function Overloading

2.4 Defining Class, Members , Object

2.5 Visibility modes

2.6 Constructors & Destructors

2.7 Friend Function

Function :-

- A function is a block of code (set of statements) which performs specific task
- It executes or runs only when it is called.
- Functions are also called methods, subroutines or procedures.
- You can pass data, known as parameters/arguments, into a function.
- A function can return data as a result .
- Functions can be called multiple times and from any location in the program.
- It saves us from writing the same code again and again. Function is written in the form of a block and instead of writing the code repeatedly you can call this function whenever you need it in the program.
- It also helps in dividing the program into well-organized segments.

Function Syntax

The syntax for creating a function is as follows:

```
return_type function_name ( parameter 1, parameter 2, . . . . )  
{  
  
    //function body  
  
}
```

Here, the return type is the data type of the value that the function will return.

Then there is the function name followed by the parameters which are not compulsory which means a function may or may not contain parameters.

Inside the function (the body), add code that defines what the function should do.

Example:-

Int sum(int,int)

Void (sum)

Call a Function

Declared functions are not executed immediately. They will be executed only when they are called.

When a function is called from the main function, then the control of the function is transferred to the function that is called. And then that function performs its task. When the task is finished, it returns the control to the main function.

To call a function, write the function's name followed by two parentheses () and a semicolon ;
Sum();

Example of program:-

```
include <iostream>
using namespace std;
void hello()
{
    cout << "hi \n";
}
int main()
{
    hello();
    hello();
    hello();
    return 0;
}
```

Output:-

hi
hi
Hi

Types of Functions in C++

There are two types of functions in C++

- Built-in functions
- User-defined functions

1. Built-in Functions:

These are functions that are already present in C++; their definitions are already provided in the header files. The compiler picks the definition from header files and uses them in the program

2. User-defined Functions:

These are functions that a user creates by themselves, wherein the user gives their own definition.

Formal And Actual Parameters :-

The parameters that are defined in the **function definition** are called **Formal Parameters**.

The parameters in the **function call** which are the actual values are called **Actual Parameters**.

Note that the type of formal and actual arguments should match. The order of formal and actual parameters should also match.

```

#include <iostream>
using namespace std;

Void swap(int a,int b)           //a and b are formal parameters
{
    b=a+b;
    a=b-a;
    b=b-a;
    cout<<"\n after Swapping : ";
    cout<<"\n a = "<<a;
    cout<<"\t b= "<<b;
    return;
}

int main()
{
    Int a,b;
    cout<<"Enter two numbers to be swapped : ";
    cin>>a>>b;
    cout<<"a = "<<a;
    cout<<"\n b= "<<b;
    swap(a,b);                  //here a and b are actual parameters
}

```

Output:

```

Enter the two numbers to be swapped: 5 3
a = 5 b = 3
After swapping: a = 3 b = 5

```

Return Values :-

Once the function performs its assigned task, it can return the result to the calling function. For this, we need the return type of the function. The function can return a single value to the calling function. The return type of the function is declared along with the function's definition. Use void to indicate that the function does not return a value.

```

#include <iostream>
using namespace std;

int sum(int a,int b)
{
    return (a+b);
}

```

```

int main()
{
    int a,b,result;

    cout<<"Enter the two numbers to be added : ";
    cin>>a>>b;

    result=sum(a,b);

    //Function call and after execution of function sum the resulted value return to the
    variable result.
    cout<<"\n Sum of the two numbers : "<<result;

}

```

Output:

Enter the two numbers to be added: 11 11
Sum of the two numbers: 22

Function Call Methods :-

You can invoke or call a function in two ways: call by value and call by function

Call by value	Call by reference
A copy of the value is passed to the function.	An address of value is passed to the function
Changes made inside the function are not reflected on other functions	Changes made inside the function are reflected outside the function as well
Actual and formal arguments will be created at different memory location	Actual and formal arguments will be created at same memory location.

Call By Value : (Pass by Value, copies of variables) :-

In this calling method, you only pass the copies of the variable and not the actual argument to the function.

As the copies of the variable or arguments are being passed, any changes made to the variable in the function don't affect the actual argument.

Due to this, the actual and formal parameters are stored at different memory locations.

Thus, changes made to formal parameters inside the function do not reflect outside the function.

Values of variables inside the main function remain as it is.

For example:

```
#include <iostream>
using namespace std;
void inc(int num1, int num2)
{
    num1= num1+20;
    num2=num2+20;
    cout<<"num1 in Inc function = " <<num1<<"\n";
    cout<<"num2 in Inc function = " <<num2<<"\n";
}

int main()
{
    int number1, number2;
    // take input
    cout << "Enter the first number: ";
    cin >> number1;
    cout << "Enter the second number: ";
    cin >> number2;

    // call add function
    inc(number1, number2);

    // displaying the result
    cout << "number1 after increasing in main function = " <<number1<<"\n";
    cout<< " number2 after increasing in main function = " << number2 << endl;

    return 0;
}
```

Output

Enter the first number: 50

Enter the second number: 60

num1 in Inc function = 70

num2 in Inc function = 80

number1 after increasing in main function = 50

number2 after increasing in main function = 60

Call By Reference: (address of actual parameters , name given to the variable)

Pass by reference is yet another technique used by C++ to pass parameters to functions. In this technique, instead of passing copies of actual parameters, we pass references to actual parameters.

This can be done by using the '&' operator.

Both actual and formal parameters refer to the same locations, so any changes made inside the function are reflected in the actual parameters of the caller.

Note: References is another name that is given to a variable.
Hence a variable and its reference share the same memory location.

In pass by reference technique, we use these references of actual parameters and as a result, **the changes made to formal parameters in the function are reflected back to the calling function.**

In this case, the actual value of the variable changes, or you can say that it reflects the changes back to the actual variable.

```
#include <iostream>
```

```
using namespace std;
```

```
// function to add two numbers
```

```
// using const references
```

```
void inc(int &num1, int &num 2)
```

```
{
```

```
    num1= num1+20;
```

```
    num2=num2+20;
```

```
    cout<<"num1 in Inc function = " <<num1<<"\n";
```

```
    cout<<"num2 in Inc function = " <<num2<<"\n";
```

```
}
```

```
int main()
```

```
{
```

```
    int number1, number2;
```

```
    // take input
```

```
    cout << "Enter the first number: ";
```

```
    cin >> number1;
```

```
    cout << "Enter the second number: ";
```

```
    cin >> number2;
```

```

    // call add function
    inc(number1, number2);

    // displaying the result
    cout << "number1 after increasing in main function = "<<number1<<"\n";
    cout<< " number2 after increasing in main function = " << number2 << endl;

    return 0;
}

```

Output:

```

Enter the first number: 50
Enter the second number: 60
num1 in Inc function = 70
num2 in Inc function = 80
number1 after increasing in main function = 70
number2 after increasing in main function = 80

```

=== Code Execution Successful ===

C++ Inline Functions

In C++, we can declare a function as inline.

This copies the code of the function to the location of the function call in compile-time and may make the program execution faster.

The code written in the function gets copied to the calling function and directly output generated which makes program execution faster.

1. Syntax of Inline Function

Syntax:

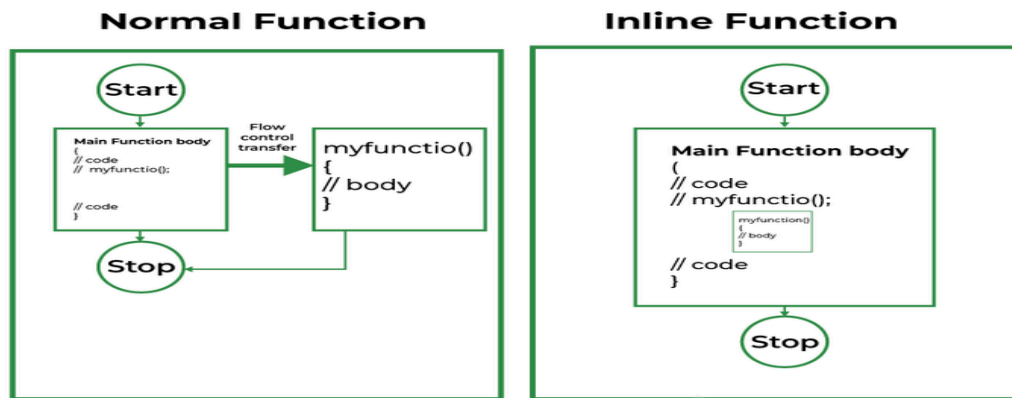
```

inline return-type function-name(parameters)
{
    // function code
}

```

To define an inline function, use the inline keyword before the function definition.

2. How Inline Functions Work



The compiler replaces the function call with the function's code at compile time. This reduces the overhead of a function call.

Use inline for small, frequently called functions to improve performance.

Avoid it for large, complex, or recursive functions to prevent excessive code duplication.

Inline is a suggestion, not a guarantee—the compiler makes the final decision.

```
#include <iostream>
using namespace std;
inline int cube(int s) { return s * s * s; }
int main()
{
    cout << "The cube of 3 is: " << cube(3) << "\n";
    return 0;
}
```

Output

The cube of 3 is: 27

Function overloading :-

Method overloading is a technique which allows you to have more than one function with the same function name but with different parameter lists.

When a method is called, Java decides which version of it to execute depending on the arguments given.

Method overloading can be possible on the following basis:

1. The number of parameters passed to the function.

2. The data type of the parameters passed to the function.

1) Method Overloading: changing no. of arguments

In this method we are defining the functions with the same name but different number of arguments.

```
#include <iostream>
using namespace std;

// function with 2 parameters
void display(int var1)
{
    cout << "Integer number: " << var1<<endl;
}

// function with int type single parameter
void display(int var1,int var2)
{
    cout << "Integer number: " << var1 << endl;
    cout << "Integer number: " << var2<< endl;
}

int main()
{
    int a = 5;
    int b = 10;

    // call function with int type parameter
    display(a);

    // call function with 2 parameters
    display(a, b);

    return 0;
}
```

Output :-

Integer number: 5

Integer number: 5

Integer number: 10

=== Code Execution Successful ===

2) Method Overloading: changing data type of arguments

Method overloading in Java also allows defining the function with the same name but different data types of arguments.

Here's an example demonstrating method overloading based on the data type of arguments:

```
#include <iostream>
using namespace std;

// function with int type single parameter
void display(int var)
{
    cout << "Integer number: " << var << endl;
}

// function with double type single parameter
void display(double var)
{
    cout << "Double number: " << var << endl;
}

// function with 2 parameters
void display(int var1, double var2)
{
    cout << "Integer number: " << var1;
    cout << " and double number: " << var2 << endl;
}

int main()
{
    int a = 5;
    double b = 10.5;

    // call function with int type parameter
    display(a);

    // call function with double type parameter
    display(b);

    // call function with 2 parameters
    display(a, b);

    return 0;
}
```

Output :-

Integer number: 5

Double number: 10.5

Integer number: 5 and double number: 10.5

=== Code Execution Successful ===

Visibility modes :-

Difference between Private, Public, and Protected in C++

Private	Public	Protected
The class members declared as private can be accessed only by the functions inside the base class.	The class members declared as public can be accessed from anywhere.	The class members declared as protected can be accessed in the base class and its derived classes.
Syntax :- private :	Syntax :- public :	Syntax :- protected:
Private members are not inherited in class.	Public members can be inherited in class.	Protected members can be inherited in class.
It provides high security for its data members.	It doesn't provide any kind of security for its data.	It provides less security than private access specifier.
Private members can be accessed with the help of a friend function.	Public members can be accessed with or without the help of a friend function.	Protected members cannot be accessed through the friend function, they are inaccessible outside the class but they can be accessed by any subclass(derived class) of that class.

Access Specifiers / visibility modes :-

Data hiding is one of the important features of object-oriented programming in C++, which allows us to hide internal object details i.e. the data members, and prevents the functions of a program from directly accessing the internal representation, data members, and member functions of a class.

The access restriction to the class member functions is specified by access modifiers. Access specifiers are also known as visibility modes in C++.

Visibility modes allow us to know how the base class will be inherited by the derived class and which members of the base class will be accessible by the derived class. Access specifiers define how the members (attributes and methods) of a class can be accessed.

In C++, there are three access specifiers:

- **public** - members are accessible from anywhere.
- **private** - members cannot be accessed (or viewed) from outside the class. They are only accessible by the members of the base class and by the friend functions.
- **protected** - members cannot be accessed from outside the class, however, they can be accessed in inherited classes that is they are accessible in the base class and the derived classes.

Example :-

```
class num
{
    public: // Public access specifier
    int x; // Public attribute
    private: // Private access specifier
    int y; // Private attribute
};
```

```
int main()
{
    num n1;
    n1.x = 25; // Allowed (public)
    cout<<n1.x;
    n1.y = 50; // Not allowed (private)
    return 0;
}
```

Here error occurs because y is private and we are trying to access value of y outside class.

If we remove the statement **n1Obj.y = 50;** from above program then value of x is printed that is x=25.

Note: By default, all members of a class are `private` if you don't specify an access specifier:

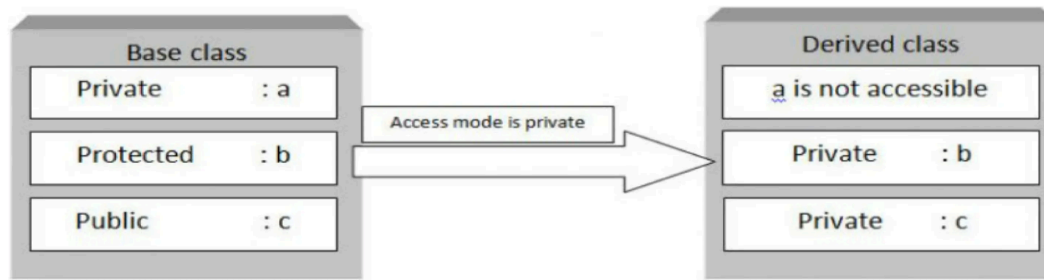
Accessibility

Accessibility	Private mode	Protected mode	Public mode
Same class	Yes	Yes	Yes
Derived class	No	Yes	Yes
Other class	No	Yes	Yes
Anywhere in the program	No	No	Yes

The Private Members

In the private visibility mode, when we inherit a child class from the parent class, then all the members (public, private and protected members) of the base class will become private in the derived class.

The following block diagram can show how data members of the base class are inherited when the derived class access mode is private.



SCALER
Topics

As you can see from the image above, the private variable `a` is not accessible in the derived class because it is of the private type inside the base class, so it is only accessible by the member functions within the base class itself.

The public variable `c` and the protected variable `b` of the base class will become private in the derived class and they will only be accessed by the member functions of the derived class.

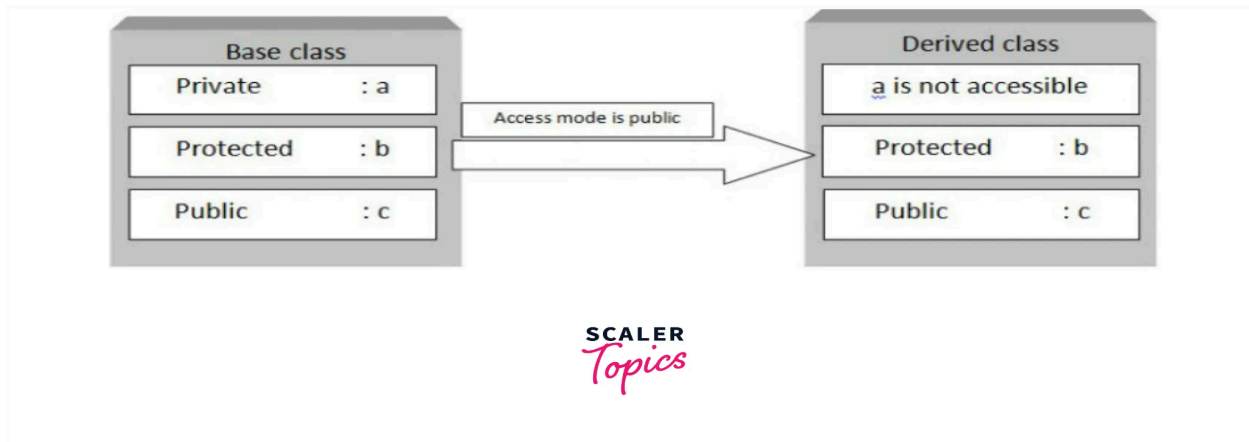
Example, suppose that on any social media app, you want status should be visible “only for me”, "Private access specifier".

The Public Members

In public visibility mode, when we inherit a child class from the parent class, then the public, private, and protected members of the base class remain public, private, and protected members respectively in the derived class as well.

The public visibility mode has the accessibility of all the members of the parent class.

The following block diagram can show how data members of the base class are inherited when the derived class access mode is public.



As you can see from the image above, the private variable `a` is not accessible in the derived class because it is of the private type, so it is only accessible by the member functions of the base class. The public variable `c` will remain public and will be accessible to the child's class and all others. The protected variable `b` of the base class will remain protected in the derived class and will be accessible only inside the derived class and its inherited classes.

The real-life example of updating the status on any social media app, we have taken above, suppose you want that your status should be visible to anyone on the internet, then, that means that you are following the concept of "Public access specifier".

Example

```
class calci
{
    public:
        int x;

        void fun()
        {
            return 2 * x;
        }
};

// main function
int main()
{
    // creating object of the derived class
    calci c1;

    // accessing public data members outside class
    C1.x = 9;

    cout << c1.x << endl;
```

```

    cout << c1.fun();
}

```

The output of the above program would be:

9

18

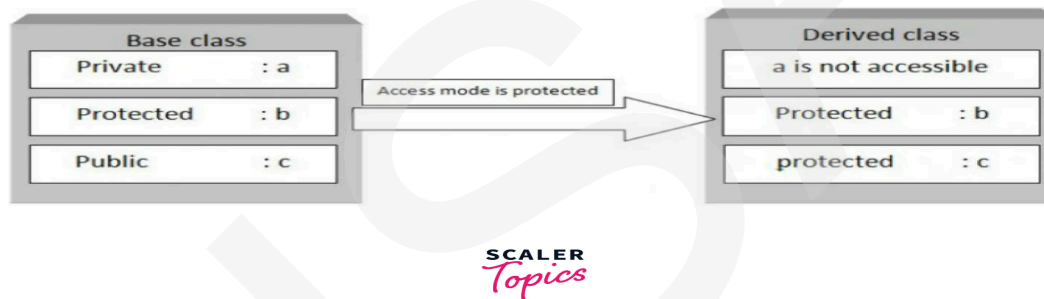
In the above program, the data member x is declared as public and so it could be accessed in any class and also from outside the class.

The Protected Members

In the protected visibility mode, when we inherit a child class from the parent class, then all the members of the base class will become the protected members of the derived class.

Because of this, these members are now only accessible by the derived class and its member functions. These members can also be inherited and will be accessible to the inherited subclasses of this derived class.

The following block diagram can show how data members of the base class are inherited when the derived class access mode is protected.



As you can see from the image above, the private variable a is not accessible in the derived class because it is of the private type, so it is only accessible by the member functions of the base class. The public variable a will become protected in the derived class and will be accessible only by the inherited subclasses of this derived class. The protected variable b of the base class will remain protected in the derived class and will be accessible only inside the derived class and its inherited classes.

the real-life example of updating the status on any social media app, if you want that your status should only be visible to your friends and your friends of friends, then, that means that you are following the concept of "Protected access specifier".

Example

```

class base
{
    protected:
        int x;

        void fun()
        {

```

```

        return 2 * x;
    }

};

// main function
int main()
{
    // creating object of the derived class
    base obj;

    // accessing public data members outside class
    obj.x = 9;

    cout << obj.x << endl;
    cout << obj.fun();
}

```

The output of the above program would be:

9

18

In the above program, the data member x is declared as protected and so it could be accessed by any subclass (derived class) of the base class.

	Derived Class	Derived Class	Derived Class
Base Class	Private Mode	Protected Mode	Public Mode
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

Constructors and Destructors in C ++ :-

A constructor in C++ is a special member function invoked automatically when an object of a class is created. It is generally used to initialize the data members of the new object. They are also used to run a default code when an object is created. The constructor in C++ has the same name as the class or structure and does not have a return value.

Remember: The access specifier of a constructor is always public.

If we have not defined a constructor in our class, then the C++ compiler will automatically create a default constructor with an empty code and no parameters.

A constructor in C++ cannot be inherited.

Syntax of a Constructor prototype

```
<class-name> (list-of-parameters);
```

A constructor in C++ can be defined in two ways:

1. inside the class

Syntax

```
<class-name> (list-of-parameters) {  
// constructor definition  
}
```

Example

```
class hello { //class  
public: //access specifier  
hello { //constructor  
// code  
}  
};
```

2. outside the class using the scope resolution operator ::

Syntax

```
<class-name>:: <class-name>(list-of-parameters){  
// constructor definition  
}
```

Example

```
Hello :: hello () { //outside the class constructor  
// code  
}
```

Types of Constructors in C++

There are three types of constructors in C++:

1. Default Constructor in C++
2. Parameterized Constructor in C++
3. Copy Constructor in C++

1. Default Constructor in C++

It is a constructor in C++ with no parameters.

Example :-

```
#include <iostream>
using namespace std;
class Student {
public:
    Student() {
        cout << "Default Constructor Invoked" << endl;
    }
};
int main() {
    Student s1; // Creating an object of Student class
    Student s2;
    return 0;
}
```

Output :-

```
Default Constructor Invoked
Default Constructor Invoked
```

The above code has a default constructor named `Student()`. When the objects `s1` and `s2` of the `Student` class is created, the constructor `Student()` gets invoked and prints the statement written inside it

2. Parameterized Constructor in C++

It is a constructor in C++ with parameters. This is the preferred method to initialize the data members of a class.

```
#include <iostream>
using namespace std;
class Student {
public:
    int id; // data member (also instance variable)
    string name; // data member (also instance variable)
    float marks;
```

```

Student(int i, string n, float m) {
    id = i;
    name = n;
    marks = m;
}

void display() {
    cout << id << " " << name << " " << marks << " " << endl;
}
};

int main() {
    Student s1 = Student(101, "Urmi", 78); // creating an object of Student
    Student s2 = Student(102, "Pragati", 89);
    s1.display();
    s2.display();
    return 0;
}

```

Output :-

```

101 Urmi 78
102 Pragati 89

```

3. Copy Constructor in C++

It is a member function that initializes the data members of an object using another object from the same class. In other words, the copy constructor in C++ is used to copy data of one object into another.

Example

```

#include <iostream>
using namespace std;
class Student {
public:
    int id; // data member (also instance variable)
    string name; // data member (also instance variable)
    float marks;

    Student(int i, string n, float m) { // parametrized constructor
        id = i;
        name = n;
        marks = m;
    }

    // Copy constructor with a Student object as a parameter

```

```

// Copies data of the obj parameter
Student(const Student& obj) {
    id = obj.id;
    name = obj.name;
    marks = obj.marks;
}

void display() {
    cout << id << " " << name << " " << marks << " " << endl;
}
};

int main() {
    Student s1 = Student(101, "Urmi", 78); // Creating an object of the Student class
    Student s2 = s1; // Copy contents of s1 to s2
    s1.display();
    s2.display();
    return 0;
}

```

In this program, we have used a copy constructor, `Student(const Student& obj)` to copy the contents of one object of the `Student` class to another.

- The parameter of this constructor has the address of an object of the `Student` class.
- We then assign the values of the variables of the `obj` object to the corresponding variables of the object calling the copy constructor. This is how the contents of the object are copied.
- The `const` in the copy constructor's parameter `const Student& obj` is used to ensure that the object being passed as an argument to the copy constructor is not modified within the copy constructor itself.
- In `main()`, we then create two objects `s1` and `s2` and then copy the contents of `s1` to `s2`.
- The `s2` object calls its copy constructor by passing the address of the `s1` object as its argument i.e. `&obj = &s1`.

Output

```

101 Urmi 78
101 Urmi 78

```

Destructor in C++ :-

Destructors in C++ are member functions in a class that deletes an object. They are called when the class object goes out of scope such as when the function ends, the program ends, a delete variable is called, etc.

Destructors don't take any argument and don't return anything. Also, destructors have the same name as their class and their name is preceded by a tilde `~`.

A destructor in C++ can be defined in two ways:

1. Inside the class

Syntax :-

```
~ <class-name>
{
// destructor definition
}
```

Example

```
class school
{ //class

    public: //access specifier
        ~school() { //destructor
            // code
        }

};
```

2. Outside the class using the scope resolution operator: :

Syntax

```
<class-name>: : ~<class-name>()
{
// destructor definition
}
```

Example

```
ScholarHat: : ~ScholarHat() { //outside the class destructor
// code
}
```

Example of Destructor in C++ Compiler

```
#include <iostream>
using namespace std;
```

```
class ScholarHat
{
    public:
        // User-Defined Constructor
        ScholarHat()
        {
```

```

        cout << "\n Constructor executed";
    }

    // User-Defined Destructor

    ~ScholarHat()
    {
        cout << "\nDestructor executed"; }
};

int main()
{
    ScholarHat t;
    return 0;
}

```

The above C++ program shows the order of execution of the constructor and destructor when an object is created and destroyed within the main() function. It defines a class ScholarHat with a user-defined constructor and destructor.

Output :-

Constructor executed
Destructor executed

Characteristics of a Destructor in C++

- **The destructor function is automatically invoked when the objects are destroyed.**
- **It cannot be declared static or const.**
- **The destructor does not have parameters.**
- **It has no return type.**
- **An object of a class with a destructor cannot become a member of the union.**
- **A destructor should be declared in the public section of the class.**
- **The programmer cannot access the address of the destructor.**

C++ Friend function :-

The friend function in C++ is defined outside the scope of the class.

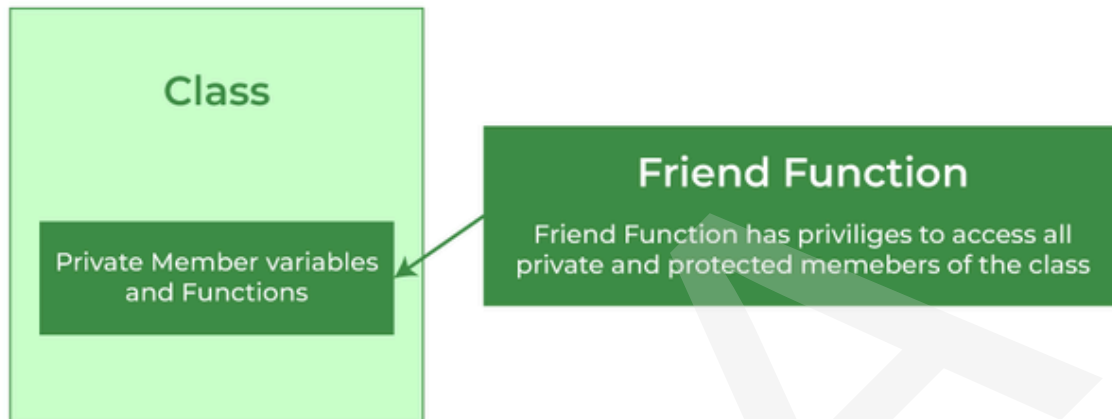
It has the authority to access all protected members and private members of the class.

If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.

By using the keyword friend compiler knows the given function is a friend function.

For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Friend Function



Declaration of friend function in C++

```
class class_name
{
    friend data_type function_name(argument/s);    // syntax of friend function.
};
```

In the above declaration, the friend function is preceded by the keyword friend. The function can be defined anywhere in the program like a normal C++ function. The function definition does not use either the keyword friend or scope resolution operator.

Characteristics of a Friend function:

1. The function is not in the scope of the class to which it has been declared as a friend.
2. It cannot be called using the object as it is not in the scope of that class.
3. It can be invoked like a normal function without using the object.
4. It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
5. It can be declared either in the private or the public part.

C++ friend function Example :-

```
#include <iostream>
using namespace std;
class Box
{
    private:
        int length = 10 ;
```

```
    public:
        friend int printLength(Box); //friend function
};
int printLength(Box b)
{
    b.length += 10;
    return b.length;
}
int main()
{
    Box b;
    cout<<"Length of box: "<< printLength(b)<<endl;
    return 0;
}
```

Output :-

Length of box: 20